Lesson 13 Teacher's Key: A Hit That Reacts

Unity - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Introducing Coroutines - Unity's built-in tool for pausing a function mid-execution using yield return, without freezing the rest of the game - genuinely new this week, and directly motivated by needing exactly this capability for the first time.

Level 1 - Full Worked Solution

- HandleHit() is a Coroutine (IEnumerator return type, started with StartCoroutine) rather than a plain function - this is required because yield return WaitForSeconds() only works inside a Coroutine.
- The explosion is instantiated, the flash sprite swapped in, hitSound played, AND RegisterKill() called, all BEFORE the yield - score/sound/wave/win-check happen at exactly the same instant as Lesson 4/7, completely unchanged. Only Destroy() moves to after the yield - that is the one and only line actually delayed.
- other.gameObject is destroyed at the very top of OnTriggerEnter2D, before StartCoroutine runs - this prevents a second overlapping bullet from double-triggering the same enemy while it's mid-Coroutine.
- hitSound now plays via Camera.main.transform.position rather than Lesson 4's original transform.position - PlayClipAtPoint() always spawns a 3D-blend temporary audio source with real distance rolloff, so a sound played at the enemy's own position can end up nearly inaudible depending on camera distance. Every other round-ending sound in this course (winSound, waveClip, gameOverSound, lifeLostSound) already used Camera.main.transform.position from the moment it was introduced; hitSound is corrected to match as of this lesson.



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

Why This Structure

- Coroutines are the right tool here specifically because the pause needs to happen INSIDE existing hit-handling logic without restructuring it across multiple objects - a much smaller change than GameMaker or TIC-80 need for the same idea.
- Giving the explosion prefab its own Destroy(gameObject, 0.3f) call in its own Start() (rather than having the Enemy manage its lifetime) keeps every object responsible for cleaning up only itself - the same discipline bullets and enemies have followed all course.
- The flash swaps sprite rather than tinting spriteRenderer.color white, for the same reason Lesson 2's Recolor swaps sprites instead of tinting them: color multiplies against the sprite's existing color, and multiplying any color by white is a no-op - our enemy sprite is opaque red, so a white color tint would produce zero visible change.
- Moving hitSound's call site to Camera.main.transform.position fixes a real, previously undocumented bug: AudioSource.PlayClipAtPoint() always creates its temporary audio source with 3D spatial blend, so distance from the AudioListener (on the camera) silently attenuates volume no matter what volumeScale argument is passed - volumeScale is a 0-1 percentage of the clip's own volume, not a boost, so it can never compensate for rolloff. Playing it at the camera's own position, exactly like every other round-ending sound already does, sidesteps the problem entirely.

Level 2 / Level 3 Talking Points

- Ask students to explain why HandleHit() needs to be IEnumerator instead of void - because yield return can only appear inside a function with that return type, a small but important C# rule worth naming explicitly.
- For students comparing engines: Godot's await and Unity's Coroutine solve the identical problem with similar underlying mechanisms, while GameMaker and TIC-80 need a genuinely different structural approach because their languages don't support pausing mid-script at all.

Common Student Mistakes

- Writing HandleHit() as a regular void function instead of IEnumerator, which won't compile once yield return is added.
- Calling HandleHit() directly instead of StartCoroutine(HandleHit()) - this also won't compile, and is an easy habit to fall into from months of calling regular functions directly.
- Forgetting to destroy the bullet before starting the Coroutine, allowing a second bullet to also trigger OnTriggerEnter2D on the same, already-dying enemy.
- Leaving hitSound's call at transform.position (Lesson 4's original placement) instead of moving it to Camera.main.transform.position - compiles and appears to work, but plays back much quieter than every other sound in the game.